PROYECTO:

ARQUITECTURA DE SOLUCIÓN

BANCA POR INTERNET – BP

Autor: Edison Casanova Yandún

Rol: Arquitecto de Soluciones

Fecha: 10 de abril de 2026

Versión 1.0

Contenido

1. Resumen ejecutivo 1
2. Marco normativo y de cumplimiento 1
   2.1. Protección de datos personales 1
   2.2. PCI – DSS 2
   2.3. ISO 27001 2
   2.4. Regulación financiera local 3
3. Contexto del negocio 3
   3.1. Necesidades clave 3
   3.2. Restricciones 3
4. Requerimientos 4
   4.1. Funcionales 4
   4.2. No funcionales 4
5. Arquitectura General 4
   5.1. Decisiones arquitectónicas 4
      5.1.1. Estilo arquitectónico 5
      5.1.2. Comunicación 5
      5.1.3. Seguridad 6
      5.1.4. Persistencia 6
      5.1.5. Consistencia 6
      5.1.6. Orquestación 7
      5.1.7. Infraestructura 7
      5.1.8. Gestión de APIs 8
      5.1.9. Monitoreo 8
      5.1.10. Prevención antifraude 9
      5.1.11. Auditoría 9
6. Arquitectura por capas 9
   6.1. Frontend (canal digital) 9
      6.1.1. Descripción general 9
      6.1.2. Principios arquitectónicos 10
      6.1.3. Aplicaciones frontend definidas 10
      6.1.3.1. Aplicación Web (SPA) 10
      6.1.3.2. Aplicación Móvil (Multiplataforma) 10
      6.1.4. Arquitectura de alto nivel 11
      6.1.5. Arquitectura de autenticación 11

6.1.6. Integración con onboarding y biometría 11
6.1.7. Seguridad en el frontend 11
6.1.8. Integración con el backend 12
6.1.9. Decisiones arquitectónicas 12
6.2. Backend (capa de negogio) 12
6.2.1. Descripción general 12
6.2.2. Principios arquitectónicos 12
6.2.3. Arquitectura de servicios 13
6.2.4. Patrones de arquitectura 13
6.2.5. Integración con sistemas externos 14
6.2.6. Gestión de auditoría 15
6.2.7. Persistencia de clientes frecuentes 15
6.2.8. Seguridad 15
6.2.9. Alta disponibilidad y resiliencia 15
6.2.10. Decisiones arquitectónicas 16
6.3. Integración 16
6.3.1. Descripción general 16
6.3.2. Componentes 16
6.3.3. Estilos de integración 17
6.3.4. Flujos de integración clave 17
6.3.5. Integración con sistemas externos 17
6.3.6. Seguridad 17
6.3.7. Alta disponibilidad y resiliencia 18
6.3.8. Patrones de integración 18
6.3.9. Decisiones arquitectónicas 19
6.4. Datos 19
6.4.1. Descripción general 19
6.4.2. Tipo de datos gestionados 20
6.4.3. Arquitectura de persistencia 20
6.4.4. Patrones de datos 20
6.4.5. Estrategia de auditoría 21
6.4.6. Persistencia para clientes frecuentes 21
6.4.7. Integración con sistemas core 21
6.4.8. Seguridad de la información 21
6.4.9. Alta disponibilidad y recuperación 22
6.4.10. Decisiones arquitectónicas 22

6.5. Seguridad transversal 22
6.5.1. Descripción general 22
6.5.2. Principios de seguridad 22
6.5.3. Arquitectura de autenticación y autorización 23
6.5.4. Gestión de identidad de acceso 23
6.5.5. Seguridad en la comunicación 23
6.5.6. Seguridad en APIs 24
6.5.7. Seguridad de datos 24
6.5.8. Auditoría y no repudio 24
6.5.9. Seguridad en infraestructura 24
6.5.10. Monitoreo y detección de amenazas 24
6.5.11. Prevención de fraude 25
6.5.12. Alta disponibilidad y seguridad 25
6.5.13. Decisiones arquitectónicas 25
7. Arquitectura Cloud 25
8. Costos 25
9. Modelo C4 26
9.1. Diagrama de contexto 26
9.2. Diagrama de contenedores 27
9.3. Diagrama de componentes 28
10. Flujos End – to – End 28
10.1. Flujo de transferencia bancaria 28
10.2. Flujo de consulta de movimientos 29
10.3. Flujo de onboarding digital 29
11. Resumen de decisiones arquitectónicas (ADR Global) 30
11.1. Introducción 30
11.2. Decisiones arquitectónicas claves 30
12. Trade – offs Aquitectónico 32

Figura 1. Diagrama de Contexto – Banca por Internet (BP) 26
Figura 2. Diagrama de Contenedores – Banca por Internet (BP) 27
Figura 3. Diagrama de Componentes – Banca por Internet (BP) 28

1. Resumen ejecutivo

La presente propuesta define la arquitectura de solución para el sistema de banca por internet de la entidad BP, cuyo objetivo es ofrecer a sus clientes una plataforma digital segura, escalable y altamente disponible para la gestión de sus productos financieros.

El sistema permitirá a los usuarios consultar el histórico de los movimientos, realizar transferencias y pagos internos e interbancarios, ejecutar y recibir notificaciones sobre los movimientos realizados, en tiempo real. Además, incorpora capacidades de onboarding digital con biometría.

La solución está basada en una arquitectura de microservicios desplegada en la nube, con integración a sistemas core existentes, y diseñada bajo principios de seguridad, resiliencia y desacoplamiento.

Los principales beneficios incluyen:

- Alta disponibilidad y tolerancia a fallos
- Escalabilidad horizontal
- Seguridad robusta basada en estándares
- Bajo acoplamiento y alta mantenibilidad

2. Marco normativo y de cumplimiento

La arquitectura propuesta incorpora controles alineados a normativas como protección de datos personales, PCI-DSS, ISO 27001, además de la regulación financiera local; implementando mecanismos de cifrado, autenticación fuerte, auditoría centralizada, segmentación de red y monitoreo continuo, garantizando la seguridad, confidencialidad, integridad y disponibilidad de la información

2.1. Protección de datos personales

Referencia: Ley Orgánica de Protección de Datos Personales (LOPDP)

Requisitos claves:

- Consentimiento explícito del usuario
- Minimización de datos
- Derecho a acceso, rectificación y eliminación
- Protección de datos sensibles

Implementación en arquitectura:

- **Capa de clasificación de datos:** Identificar, clasificar y etiquetar los datos según su nivel de sensibilidad para aplicar controles diferenciados.
- **Cifrado obligatorio:** Se define un modelo de clasificación: datos personales (novel alto); datos financieros (nivel crítico); y, datos biométricos (nivel altamente crítico)
- **Tokenización o enmascaramiento:** se evita la exposición de datos sensibles, tanto a nivel de sistema como a nivel de logs; para ello, se reemplaza los datos sensibles por tokens

- **Gestión del consentimiento:** Registrar, gestionar y validar el consentimiento del usuario para uso de datos. Se implementa funciones de registro, consulta y revocatoria de consentimiento
- **Gestión del ciclo de vida de los datos:** Gestionar el ciclo de vida de los datos. Se implementa funciones de creación, uso, almacenamiento y eliminación.

2.2. PCI – DSS

Referencia: PCI – DSS (se incluye esta referencia asumiendo que los pagos pueden realizarse a través de tarjetas)

Requisitos claves:

- Protección de datos de tarjetas
- Segmentación de red
- Control de accesos
- Monitoreo continuo

Implementación en arquitectura:

- **Segmentación de red:** Reducir el alcance del PCI, aislando los sistemas que procesan datos de tarjetas, para ello, se define una arquitectura de red con múltiples zonas: Internet → DMZ (WAF + API Gateway) → Aplicación → PCI. Además, se hace un aislamiento lógico con VPC exclusivo para PCI, subredes privadas sin acceso a internet, y grupos con niveles de seguridad restrictivos
- **Gestión del PAN:** Eliminar almacenamiento de datos sensibles de tarjeta (PAN completo) para reducir riesgo. Se reemplaza el PAN por token
- **Protección de APIs:** Las aplicaciones estarán resguardadas frente a ataques externos mediante la implementación de un WAF, evitando así la exposición directa de las APIs a internet.
- **IAM robusto:** Garantizar que cada usuario o sistema acceda solo a lo estrictamente necesario. Se establece roles por servicio y autenticación fuerte o multifactor.
- **Auditoría de accesos:** Se registra y se permite auditar todos los accesos a datos sensibles, transacciones, autenticaciones, errores. Se implementa notificaciones automáticas, especialmente en accesos sospechosos.

2.3. ISO 27001

Referencia: ISO 27001

Requisitos claves:

- Gestión de riesgos
- Control de accesos
- Seguridad en desarrollo
- Gestión de incidentes

Implementación en arquitectura:

- **IAM centralizado:** Se usa un proveedor de identidad central (IdP corporativo), un modelo de roles y segregación de funciones.

- **Seguridad por diseño:** Se implementa controles de seguridad en cada etapa, se realizan pruebas de análisis estático (análisis de código fuente), dinámico (aplicación en ejecución, simulación de ataques reales), se revisa dependencias y se establece contenedores seguros.
- **Gestión de secretos:** Se implementa controles para proteger credenciales, claves, tokens. Se gestiona y clasifica los secretos por tipo, los microservicios obtienen secretos en runtime (credencial temporal), se implementa un vault centralizado.
- **Registro de auditoría:** Se garantiza la trazabilidad completa y evita alteración de registros. Los logs se centralizan y se implementa almacenamiento inmutable para impedir que los objetos sean borrados o sobrescritos durante un tiempo definido.

2.4. Regulación financiera local

Referencia: Norma de control de riesgo operativo y de la seguridad de la información de la superintendencia de bancos (Ecuador)

Requisitos claves:

- Trazabilidad de transacciones
- Auditoría completa
- Disponibilidad del sistema
- Protección contra fraude

Implementación en arquitectura:

- **Auditoría basada en eventos:** Se implementa un modelo basado en eventos, para Garantizar trazabilidad completa, cronológica e inalterable de todas las operaciones del cliente.
- **Alta disponibilidad:** Se implementa balanceadores de carga, despliegues en múltiples zonas, escalado automático, failover automático y replicación de bases de datos para garantizar continuidad del servicio sin interrupciones
- **Detección de fraude:** Se implementa microservicios dedicados a la detección de fraudes, con validaciones en tiempo real para realizar acciones de bloqueo de transacción, solicitudes de autenticación multifactor; y, alertas en tiempo real para detectar y prevenir fraudes antes de que ocurran
- **No repudio:** Se incluye hash de la operación y firma digital en cada transacción, para garantizar que una transacción no pueda ser negada ni alterada por el usuario

3. Contexto del negocio

3.1. Necesidades clave

- Banca por internet
- Consulta de movimientos
- Transferencias
- Pagos

3.2. Restricciones

- Regulación financiera

- Integración con sistemas Core
- Notificaciones obligatorias

4. Requerimientos

4.1. Funcionales

- **Consulta de movimientos:** Habilita la consulta de transacciones históricas y recientes asociadas a un producto financiero
- **Transferencias internas:** Permite la transferencia de fondos entre cuentas del mismo banco en tiempo casi real
- **Transferencias interbancarias:** Facilita transferencias hacia otras instituciones financieras mediante integración con redes externas
- **Pagos:** Permite realizar pagos de servicios, tarjetas, impuestos u otros convenios
- **Onboarding con reconocimiento facial:** Proceso de alta de nuevos clientes 100% digital, incluyendo captura de datos, validación de identidad y biometría

4.2. No funcionales

- **Alta disponibilidad:** Se garantiza continuidad operativa >= 99.9%, mediante despliegue en múltiples zonas de disponibilidad, balanceadores de carga y mecanismos failover automático, réplicas de datos y monitoreo proactivo
- **Seguridad:** Se implementa un modelo basado en OAuth2 / OpenID Connect para autenticación y autorización con emisión de tokens. Toda la comunicación será cifrada mediante TLS 1.2+ en tránsito y ciifrado AES-256 en reposo. Se implementa gestión de identidades (IAM) y autenticación multifactor (MFA)
- **Tolerancia a fallos:** Operación continua ante fallos parciales mediante implementación de patrones como circuit breaker, retries con backoff exponencial y colas de mensajería para desacoplar componentes. Además, se Implementa microservicios con aislamiento de fallos y uso de eventos para evitar dependencias síncronas críticas.
- **Escalabilidad:** La plataforma escala horizontalmente de forma automática según la demanda, tanto a nivel de servicios como de base de datos. Se considera la implementación de contenedores (kubernetes), particionamiento de datos (sharding), cachés distribuidos (Redis) y CDN para contenido estático, garantizando elasticidad sin degradación del servicio
- **Baja latencia:** Tiempo de respuesta óptimo (idealmente < 300 ms) en operaciones críticas) mediante optimización de consultas, uso de caché, proximidad geográfica y minimización de llamadas síncronas. Se implementa patrones CQRS y lecturas desde réplicas para mejorar la experiencia del usuario.
- **Auditoría completa:** Todas las acciones y transacciones se registran de forma trazable, inmutable y segura, incluyendo quién, cuándo y qué operación se ejecutó. Se implementa mediante logging centralizado, event sourcing, con integración a sistemas SIEM para monitoreo y cumplimiento regulatorio (no repudio).

5. Arquitectura General

5.1. Decisiones arquitectónicas

Las decisiones arquitectónicas de la solución “banca por internet” se fundamentan en principios de diseño de sistemas distribuidos, resiliencia operativa y cumplimiento normativo del sector

financiero. Cada decisión, desde el estilo arquitectónico hasta los mecanismos de seguridad, comunicación y persistencia, responden a la necesidad de garantizar alta disponibilidad, escalabilidad, seguridad y trazabilidad de las transacciones. Las decisiones tomadas siguen principios clave: **Desacoplamiento** (microservicios + eventos); **Resiliencia** (tolerancia a fallos + arquitectura reactiva); **Seguridad** (OAuth2 + cifrado); **Escalabilidad** (cloud + contenedores); y, **Gobernanza** (API Gateway + observabilidad).

5.1.1. Estilo arquitectónico

Decisión: Adaptar una arquitectura de microservicios; permite desacoplar dominios de negocio, escalar de forma independiente, mejorar la resiliencia del sistema y alinearse con prácticas modernas como DevOps y cloud-native. Esta decisión garantiza una plataforma más flexible, robusta y preparada para cumplir con los estándares del sector financiero

Justificación:

- **Principio de desacoplamiento y cohesión alta**: Los microservicios permiten encapsular capacidades de negocio (pagos, transferencias, onboarding), reduciendo el acoplamiento y facilitando mantenibilidad.
- **Escalabilidad independiente**: Permite escalar componentes críticos (ej. transferencias) sin impactar todo el sistema, alineado con el principio de escalabilidad horizontal.

Opciones evaluadas: Se evaluó varias alternativas como monolito tradicional, monolito modular, SOA y serverless. Sin embargo, estas opciones fueron descartadas principalmente por limitaciones en escalabilidad, resiliencia y capacidad de evolución.

Por ejemplo, el monolito presenta un alto acoplamiento y no permite escalar componentes de forma independiente, lo cual es crítico en escenarios como picos de transacciones. El monolito modular mejora la organización, pero mantiene restricciones en despliegue y operación. SOA, por su parte, introduce dependencia en un bus central, lo que puede afectar la disponibilidad y generar cuellos de botella; y, serverless puro no ofrece el nivel de control transaccional y latencia predecible requerido para operaciones financieras críticas.

5.1.2. Comunicación

Decisión: Adoptar un modelo híbrido APIs REST + mensajería / eventos (event-driven); permite consistencia inmediata donde es crítico, habilita resiliencia y desacoplamiento mediante eventos, mejora la experiencia del usuario con respuestas rápidas; y, soporta escalabilidad y procesamiento distribuido.

Justificación:

- **Trade – off consistencia vs disponibilidad (Teorema CAP)**: REST permite consistencia inmediata en operaciones críticas, mientras que eventos permiten alta disponibilidad y resiliencia.
- **Patrón de arquitectura reactiva**: El uso de eventos desacopla productores y consumidores, mejorando tolerancia a fallos y elasticidad.

Opciones evaluadas: Se descartaron las opciones individuales como solo REST síncrono o solo eventos, porque no cubren estos requisitos de forma integral. Un modelo solo síncrono genera alto acoplamiento y riesgo de fallos en cascada; un modelo solo asíncrono no garantiza respuesta inmediata ni consistencia fuerte; el uso de base de datos compartida introduce acoplamiento a nivel de datos; y un ESB centralizado genera cuellos de botella y reduce la flexibilidad.

5.1.3. Seguridad

Decisión: Implementar OAuth2 + OIDC con tokens JWT; es la mejor opción, ya que garantiza seguridad, interoperabilidad y alineación con estándares modernos del sector financiero.

Justificación:

- **Delegación de autorización (OAuth2)**: Permite separar autenticación de autorización, habilitando integración segura con terceros (Open Banking). La institución cuenta con un producto que puede ser configurado para este fin, por lo que, se simplifica la implementación porque no requiere la implementación de toda la lógica.
- **Arquitectura stateless**: JWT habilita una arquitectura stateless, reduciendo la dependencia de sesiones y mejorando el rendimiento en entornos distribuidos.

Opciones evaluadas: Se descartaron otras opciones como autenticación basada en sesión o API Keys, porque no cumplen con los requisitos de seguridad y escalabilidad. La autenticación basada en sesiones introduce estado en el servidor, dificultando el escalado horizontal. Las API Keys no son adecuadas para escenarios con usuarios finales, ya que no gestionan identidad ni permisos de forma granular. Los mecanismos propietarios o básicos carecen de estandarización y dificultan la integración con terceros, algo clave en ecosistemas financieros.

5.1.4. Persistencia

Decisión: Cada microservicio gestiona su propia base de datos; la separación de bases de datos por microservicio es la mejor opción, ya que permite independencia, resiliencia y evolución continua en un entorno bancario.

Justificación:

- **Principio de autonomía de servicios:** Evita acoplamiento a nivel de datos, permitiendo evolución independiente.
- **Patrón Database per Service:** Reduce el riesgo de cambios colaterales y mejora la resiliencia organizacional del sistema.

Opciones evaluadas: Se descartaron otras opciones como bases de datos o esquemas compartidos, porque no son viables en arquitecturas distribuidas. Una base de datos compartida genera alto acoplamiento, ya que múltiples servicios dependen del mismo esquema, dificultando cambios y despliegues independientes. Además, incrementa el riesgo de inconsistencias y conflictos por concurrencia. Por otro lado, un esquema compartido, aunque segmentado, sigue manteniendo dependencias estructurales que limitan la escalabilidad y la autonomía de los equipos.

5.1.5. Consistencia

Decisión: Adoptar consistencia eventual en procesos no críticos; la consistencia fuerte es esencial en operaciones críticas; y, se puede tener consistencia eventual en procesos no críticos, de esta manera se logra un balance óptimo entre rendimiento, resiliencia y control transaccional.

Justificación:

- **Teorema CAP**: En sistemas distribuidos, priorizar disponibilidad y tolerancia a particiones implica sacrificar consistencia inmediata.
- **Escalabilidad y performance**: Reduce bloqueos y permite mayor rendimiento (throughput) en sistemas de alta concurrencia.

Opciones evaluadas: Se descartó aplicar consistencia fuerte en todos los casos porque introduce bloqueos, mayor latencia y limita la escalabilidad, afectando el rendimiento bajo alta concurrencia. Además, aumenta la complejidad operativa en sistemas distribuidos. Por otro lado, un modelo completamente eventual para todas las operaciones tampoco es viable, ya que en procesos críticos como transferencias financieras se requiere confirmación inmediata y garantías ACID.

5.1.6. Orquestación

Decisión: Orquestación para procesos críticos (transferencias) y coreografía para eventos secundarios; el enfoque híbrido es el más adecuado, ya que combina control donde es necesario y desacoplamiento donde es posible, alineándose con los requerimientos de un sistema bancario moderno.

Justificación:

- **Control vs desacoplamiento**: La orquestación garantiza un flujo controlado, manejo explícito de estados y trazabilidad, lo cual es clave en un entorno financiero.
- **Escalabilidad organizacional**: La coreografía permite que eventos secundarios, como notificaciones o auditoría, se gestionen de forma desacoplada, mejorando la escalabilidad y resiliencia.

Opciones evaluadas: Se descartó usar solo orquestación porque introduce un punto central de control que puede convertirse en cuello de botella y reducir la flexibilidad del sistema. Por otro lado, un enfoque solo basado en coreografía no es viable para procesos críticos, ya que pierde control explícito del flujo, dificulta la gestión de errores y complica la trazabilidad en transacciones financieras.

5.1.7. Infraestructura

Decisión: Despliegue en contenedores orquestados con Kubernetes; Kubernetes se posiciona como la mejor opción, al proporcionar un entorno robusto, flexible y alineado con arquitecturas cloud – native.

Justificación:

- **Portabilidad y estandarización**: Contenedores garantizan consistencia entre ambientes (dev, QA, prod), permitiendo portabilidad, escalabilidad automática y resiliencia en entornos distribuidos
- **Autoescalado y resiliencia**: Kubernetes facilita el autoescalado, balanceo de carga, recuperación automática y despliegues continuos, lo cual es clave para una banca por internet que requiere alta disponibilidad y operación.

Opciones evaluadas: Se descartaron otras opciones porque presentan limitaciones. El uso de máquinas virtuales implica mayor sobrecarga operativa y menor eficiencia en el uso de recursos, además de tiempos más lentos de aprovisionamiento. Un PaaS cerrado simplifica la operación, pero reduce el control sobre la infraestructura y puede generar dependencia del proveedor, lo cual es crítico en el sector financiero. Por otro lado, ejecutar contenedores sin un orquestador no es viable a gran escala, ya que dificulta la gestión de alta disponibilidad, escalabilidad y recuperación ante fallos.

5.1.8. Gestión de APIs

Decisión: Implementar un API Gateway como punto de entrada; proporciona un punto de control unificado, mejora la seguridad y habilita una gestión eficiente y escalable de los servicios.

Justificación:

- **Patrón API Gateway**: Permite centralizar funciones transversales como autenticación, autorización, rate limiting, monitoreo y versionado de APIs; además de abstraer la complejidad de los microservicios hacia los canales digitales.
- **Seguridad perimetral**: Reduce superficie de ataque al no exponer servicios internos directamente, esto mejora la seguridad, la gobernanza y facilita la evolución del sistema sin impactar a los consumidores.

Opciones evaluadas: Se descartaron otras opciones porque no son viables a escala. Exponer los microservicios directamente incrementa la superficie de ataque, dificulta la gestión de seguridad y genera acoplamiento con los clientes. Un enfoque sin gateway también complica la implementación de políticas comunes (como control de tráfico o auditoría). Por otro lado, soluciones como un proxy básico o balanceador de carga no cubren capacidades avanzadas de gestión de APIs requeridas en el sector financiero.

5.1.9. Monitoreo

Decisión: Implementar monitoreo completo (logs, métricas, trazas distribuidas); permite monitoreo proactivo, diagnóstico eficiente y cumplimiento de altos estándares de disponibilidad y calidad del servicio.

Justificación:

- **Principios SRE (Site Reliability Engineering)**: No se puede operar lo que no se puede medir, por lo que, es fundamental tener visibilidad “end – to – end” del sistema. Esto permite detectar fallos, analizar el rendimiento, entender el comportamiento de las transacciones y reducir tiempos de diagnóstico.
- **Sistemas distribuidos**: El tracing distribuido permite diagnosticar fallos en flujos complejos entre microservicios.

Opciones evaluadas: Se descartaron otras opciones porque no son suficientes. Un enfoque basado solo en logs no permite entender el rendimiento ni correlacionar eventos entre servicios. Usar únicamente métricas limita el análisis detallado de errores; y, no contar con trazabilidad distribuida dificulta seguir el flujo de una transacción a través de múltiples servicios, lo cual es crítico en banca.

Indicadores clave: Se considera las siguientes métricas para medir y garantizar la confiabilidad en el servicio conforme las expectativas del negocio y las regulaciones financieras:

INDICADORES	ALERTAS	HERRAMIENTAS
• Latencia de APIs (< 300 ms) • Tasa de error (< 1%) • Disponibilidad (> 99.9%)	• Fallos en transferencias • Errores de autenticación • Caída de servicios críticos	• Prometheus / CloudWatch • OpenTelemetry • Dashboards en tiempo real

5.1.10. Prevención antifraude

La arquitectura incorpora mecanismos detección de fraude:

MECANISMO	JUSTIFICACIÓN
• Validación en tiempo real de transacciones • Reglas de negocio (límites, comportamiento inusual) • Análisis de patrones de usuario	• Reducción de riesgos financieros • Protección del cliente • Cumplimiento regulatorio

5.1.11. Auditoría

Decisión: Implementar auditoría basada en event sourcing + almacenamiento inmutable; asegura evidencia confiable, auditabilidad completa y cumplimiento normativo.

Justificación:

- **No repudio**: Los eventos firmados garantizan integridad y trazabilidad.
- **Reproducibilidad del estado**: permite registrar cada evento de negocio como una secuencia inalterable, facilitando la reconstrucción del estado del sistema y cumpliendo con requerimientos regulatorios del sector financiero.

Opciones evaluadas: Se descartaron otras opciones porque no ofrecen el mismo nivel de garantías. Los logs tradicionales pueden ser modificados o no capturan el contexto completo de negocio. Las auditorías basadas en tablas históricas suelen ser parciales y dependen del diseño de cada sistema, lo que dificulta la trazabilidad "end – to – end". Además, los enfoques sin inmutabilidad no garantizan integridad frente a manipulaciones, lo cual es crítico en entornos bancarios.

6. Arquitectura por capas

La arquitectura de banca por internet se basa en una separación clara de responsabilidades, donde:

- Frontend: experiencia del cliente
- Backend: lógica de negocio
- Integración: interoperabilidad
- Datos: consistencia e información
- Seguridad: protección transversal

A continuación, se analiza cada capa, considerando no solo su función sino también su rol dentro de una arquitectura moderna, regulada y de alta criticidad.

6.1. Frontend (canal digital)

6.1.1. Descripción general

La capa Frontend de la solución BP se define como un canal digital moderno, seguro y desacoplado, que garantiza una experiencia de usuario eficiente y confiable, integrándose de forma robusta con los servicios backend y cumpliendo con los estándares de seguridad, disponibilidad y regulación del sector financiero, permitiendo la ejecución de operaciones como:

- Consulta de movimientos históricos
- Transferencias entre cuentas propias e interbancarias
- Pagos

La solución contempla dos aplicaciones principales:

- Aplicación Web tipo SPA (Single Page Application)
- Aplicación móvil desarrollada en framework multiplataforma

Ambas aplicaciones operan como clientes desacoplados que consumen servicios backend a través de una capa de integración basada en API Gateway, garantizando seguridad, escalabilidad y mantenibilidad.

6.1.2. Principios arquitectónicos

La capa Frontend se diseña bajo los siguientes principios:

- Desacoplamiento total mediante consumo de APIs
- API-first para estandarización e interoperabilidad
- Seguridad por diseño (Security by Design) alineada a normativa financiera
- Experiencia de usuario optimizada (UX-first) para canales digitales
- Alta disponibilidad (HA) mediante infraestructura distribuida
- Arquitectura escalable y resiliente
- Soporte omnicanal (web y móvil)

6.1.3. Aplicaciones frontend definidas

6.1.3.1. Aplicación Web (SPA)

TENOLOGÍAS EVALUADAS	TECNOLOGÍA RECOMENDADA	JUSTIFICACIÓN
• Angular • React	• React	• **Desempeño y experiencia fluida:** Navegación sin recarga (SPA); y, mejora la interacción en operaciones críticas (transferencias, pagos) • **Desacoplamiento del backend:** Permite evolución independiente; y, facilita integración con múltiples servicios • **Escalabilidad:** Modularización por dominios (cuentas, pagos, transferencias)

6.1.3.2. Aplicación Móvil (Multiplataforma)

TENOLOGÍAS EVALUADAS	TECNOLOGÍA RECOMENDADA	JUSTIFICACIÓN
• Flutter • React Native	• Flutter	• **Alto rendimiento cercano a nativo:** Ideal para operaciones sensibles como autenticación biométrica • **Consistencia visual en iOS y Android:** Reduce costos de desarrollo • **Soporte robusto para integración con hardware:** Biometría (huella, reconocimiento facial)

6.1.4. Arquitectura de alto nivel

La capa Frontend opera bajo el siguiente flujo:

- El usuario accede vía web o app móvil
- El tráfico pasa por un WAF para protección contra amenazas
- El contenido estático se distribuye mediante CDN
- Las solicitudes se enrutan hacia un API Gateway
- El API Gateway gestiona autenticación, autorización y enrutamiento

6.1.5. Arquitectura de autenticación

Ambas aplicaciones delegan la autenticación a un proveedor de identidad existente basado en OAuth 2.0 / OpenID Connect. Se implementa flujo de authorization code flow con PKCE

Justificación:

- Mayor seguridad para clientes públicos (SPA y aplicación móvil)
  - Evita exposición de secretos
  - Protege contra ataques de interceptación
- Cumplimiento de estándares modernos de seguridad
  - Recomendado por OAuth 2.1
  - Alineado a buenas prácticas bancarias

6.1.6. Integración con onboarding y biometría

La aplicación móvil incorpora un flujo de onboarding digital con reconocimiento facial.

- **Componentes involucrados:**
  - SDK de biometría facial (ej. AWS Rekognition, Azure Face API)
  - Servicio de validación de identidad
  - Integración con proveedor de autenticación
- **Flujo:**
  - Captura de rostro
  - Validación contra documento/registro
  - Creación de identidad digital
  - Registro en sistema de autenticación
- **Métodos de autenticación posteriores:**
  - Usuario y contraseña
  - Huella digital
  - Reconocimiento facial

6.1.7. Seguridad en el frontend

- **Controles:**
  - Protección contra XSS y CSRF
  - Content Security Policy (CSP)
  - Validación y sanitización de entradas
  - Uso obligatorio de HTTPS (TLS 1.2+)
- **Protección de sesión:**
  - Cookies seguras (HttpOnly, Secure)
  - No almacenamiento de datos sensibles en cliente

6.1.8. Integración con el backend

El frontend consume servicios a través del API Gateway, el cual expone:

- Servicio de consulta de datos básicos (existe)
- Servicio de consulta de movimientos (existe)
- Servicio de consulta de transferencias (existe)
- Servicio de consulta de perfil consolidado del cliente (nuevo)
- Servicio de historial optimizado (nuevo)

Justificación:

- Reduce latencia en frontend
- Minimiza llamadas a múltiples sistemas backend

6.1.9. Decisiones arquitectónicas

DECISIÓN	JUSTIFICACIÓN
Uso de SPA	Mejora UX + desacoplamiento
Flutter para móvil	Rendimientos + biometría
OAuth2 con PKCE	Seguridad en clientes públicos
API Gateway	Centralización de acceso
CDN	Optimización del rendimiento

6.2. Backend (capa de negogio)

6.2.1. Descripción general

La capa Backend constituye el núcleo de procesamiento de la solución de banca por internet de la entidad BP, encargada de gestionar la lógica de negocio, la orquestación de servicios, la integración con sistemas internos y externos, así como la persistencia y auditoría de la información. Esta capa se diseña bajo un enfoque de microservicios desacoplados, permitiendo escalabilidad, resiliencia y evolución independiente de los componentes

Esta capa soporta funcionalidades críticas como:

- Consulta de información del cliente
- Consulta de movimientos históricos
- Ejecución de transferencias y pagos
- Gestión de autenticación y autorización
- Generación de eventos y notificaciones

6.2.2. Principios arquitectónicos

- Arquitectura basada en microservicios
- Desacoplamiento funcional por dominio (DDD)
- Event – driven architecture (EDA)
- Alta disponibilidad (HA) y tolerancia a fallos
- Escalabilidad horizontal
- Resiliencia (circuit breakers, retries)
- Seguridad por diseño
- Observabilidad completa

6.2.3. Arquitectura de servicios

TIPO DE SERVICIO	SERVICIO	FUNCIÓN	JUSTIFICACIÓN
Servicio base existentes	Clientes	Gestiona información básica del cliente (Core)	
	Movimientos	Consulta de transacciones históricas	
	Transferencias	Orquesta transferencias internas e interbancarias	
Servicios adicionales a implementar	Perfil consolidado	Agregar información del Core + sistema complementario	• Reduce latencia en frontend (menos llamadas) • Implementa patrón Backend for Frontend (BFF) o agregador
	Notificaciones	Gestiona envío a múltiples canales (email, SMS, push)	• Cumplimiento normativo (mínimo 2 canales) • Desacopla lógica de notificación del core transaccional
	Auditoría	Registro de todas las acciones del cliente	• Cumplimiento regulatorio (trazabilidad) • Soporte para no repudio
	Autenticación	Integración con proveedor OAuth2 existente y gestión de tokens y sesiones	

6.2.4. Patrones de arquitectura

PATRÓN	APLICACIÓN	JUSTIFICACIÓN
Microservicios		• Escalabilidad independiente • Despliegue desacoplado
API Gateway	Punto único de entrada	• Centraliza seguridad • Simplifica el consumo desde el frontend
Backend for Frontend (BFF)	Servicio de perfil consolidado	• Optimiza respuestas al cliente • Reduce complejidad en el frontend
Arquitectura basada en eventos (EDA)	Notificaciones Auditoría	• Desacoplamiento entre servicios • Alta escalabilidad
Segregación de responsabilidad de comandos y consultas (CQRS)	Lectura de movimientos vs escritura de transacciones	• Optimización de consultas • Mejora rendimiento

Integración con sistemas externos	El backend se integra con: • Core bancario (Datos básicos, movimientos) • Sistema complementario (Información detallada del cliente) • Sistema de notificaciones (Email, SMS / push)	• Aisla cambios en sistemas legacy • Mejora mantenibilidad
Gestión de auditoría	• **Componentes:** Event Bus (Kafka / SNS-SQS), Servicio de Auditoría, Base de datos de auditoría (append-only) • **Flujo:** Usuarios ejecuta acción, servicio genera evento, evento publica en bus, servicio de auditoría lo persiste	• Garantiza trazabilidad completa • Soporta cumplimiento normativo • Permite reconstrucción de eventos
Persistencia de clientes frecuentes	• **Componentes:** Redis (cache), bases de datos optimizadas para lectura	• Reduce latencia en consultas frecuentes • Disminuye carga en sistemas Core
Alta disponibilidad y resiliencia	• **Estrategia:** Despliegue en múltiples zonas (Multi-AZ); Balanceadores de carga; escalamiento automático; Circuit breakers; políticas de reintento • **Recuperación ante desastres (DR):** Réplica de bases de datos; backups automáticos; estrategia multi región	

6.2.5. Integración con sistemas externos

SISTEMA	ESTRETEGIA DE INTEGRACIÓN	JUSTIFICACIÓN
Core bancario: • Datos básicos • Movimientos • Productos **Sistema complementario:** • Información detallada del cliente	• Uso de adaptadores (pattern Adapter) • Abstracción de sistemas externos	• Aisla cambios en sistemas legacy • Mejora mantenibilidad

Sistema de notificaciones: • Email • SMS / Push		

6.2.6. Gestión de auditoría

SOLUCIÓN	COMPONENTES	FLUJO	JUSTIFICACIÓN
Event Sourcing + Logging Centralizado	• Event Bus (Kafka / SNS-SQS) • Servicio de Auditoría • Base de datos de auditoría (append-only)	• Usuario ejecuta acción • Servicio genera evento • Evento se publica en bus • Servicio de auditoría lo persiste	• Garantiza trazabilidad completa • Soporta cumplimiento normativo • Permite reconstrucción de eventos

6.2.7. Persistencia de clientes frecuentes

PATRÓN	COMPONENTES	JUSTIFICACIÓN
Cache + Read Model (CQRS)	• Redis (cache) • Base de datos optimizada para lectura	• Reduce latencia en consultas frecuentes • Disminuye carga en sistemas Core

6.2.8. Seguridad

Para garantizar la seguridad a nivel backend, se implementa los siguientes controles:

- Validación de tokens OAuth2
- Autorización basada en roles
- Encriptación de datos sensibles
- Protección contra ataques (rate limiting)
- Comunicación segura (HTTPS/TLS) (seguridad en servicio)
- Mutual TLS entre microservicios (seguridad en servicio)

6.2.9. Alta disponibilidad y resiliencia

CAPACIDAD	ESTRATEGIA
Disponibilidad	• Despliegue en múltiples zonas (Multi-AZ) • Balanceadores de carga • Escalado automático • Circuit breakers • Políticas de reintento
Recuperación ante desastres	• Replicación de bases de datos • Backups automáticos • Estrategia multi región

6.2.10. Decisiones arquitectónicas

DECISIÓN	JUSTIFICACIÓN
Microservicios	Escalabilidad + desacoplamiento
Event – driven	Resiliencia + auditoría
CQRS	Optimización lectura / escritura
API Gateway	Seguridad centralizada
Cache (Redis)	Mejora de rendimiento

6.3. Integración

La capa de integración se define como un habilitador clave de la arquitectura, permitiendo la comunicación segura, desacoplada y resiliente entre los distintos componentes del sistema, garantizando el cumplimiento de los requerimientos funcionales y normativos del sector financiero, así como la escalabilidad y evolución futura de la solución.

6.3.1. Descripción general

La capa de integración es responsable de habilitar la comunicación segura, eficiente y desacoplada entre los distintos componentes de la solución "Banca por Internet". La capa de integración está diseñada para garantizar: interoperabilidad, desacoplamiento, escalabilidad, resiliencia y seguridad; incluyendo:

- Canales digitales (Frontend)
- Servicios backend (microservicios)
- Sistemas core bancarios
- Sistemas externos (notificaciones, biometría, autenticación)

6.3.2. Componentes

COMPONENTE	APLICACIÓN	JUSTIFICACIÓN
API Gateway (Punto único de entrada para todas las solicitudes del frontend)	• Enrutamiento de solicitudes • Validación de tokens OAuth2 • Control de acceso • Rate limiting • Logging y monitoreo	
Capa de Orquestación (Backend Services)	• Orquesta las llamadas a múltiples servicios • Implementa la lógica de negocio compuesta	
Adaptadores de integración **Patrón aplicado:** Adapter Pattern	• Conectar con sistemas externos: Core bancario, Sistema complementario, Sistemas de notificación	• Aísla dependencias externas • Permite cambios sin afectar el core del sistema
Bus de Eventos / Mensajería **Tecnologías:** Kafka / AWS SNS – SQS / Azure Service Bus	• Notificaciones • Auditoría • Procesos asíncronos	

6.3.3. Estilos de integración

ESTILO	APLICACIÓN	JUSTIFICACIÓN
Integración síncrona: APIs REST	• Consultas de cliente • Movimientos • Validaciones	• Respuesta inmediata • Simplicidad
Integración asíncrona: Event – Driven	• Notificaciones • Auditoría • Procesos no críticos	• Desacoplamiento • Alta escalabilidad • Resiliencia

6.3.4. Flujos de integración clave

PROCESO	FLUJO
Consulta de información del cliente	Frontend → API Gateway API Gateway → Servicio de Perfil Servicio de Perfil → Core + Sistema Complementario Respuesta consolidada → Frontend
Transferencias	Frontend → API Gateway API Gateway → Servicio de Transferencias Servicio → Core Bancario Evento generado → Bus de eventos Notificación enviada
Notificaciones	Evento de transacción Publicación en bus Servicio de notificaciones consume evento Envío por múltiples canales: Email, SMS / Push

6.3.5. Integración con sistemas externos

SISTEMA EXTERNO	ESTRATEGIA DE INTEGRACIÓN
Core bancario	• Integración mediante APIs o servicios legacy • Uso de adaptadores para desacoplamiento
Sistema complementario	• Consulta de información enriquecida
Sistema de notificación	• Failover entre proveedores • Envío redundante (por normativa se debe tener mínimo dos proveedores)
Biometría (Onboarding)	• Integración con APIs externas para el reconocimiento facial

6.3.6. Seguridad

Se implementa los siguientes controles:

- Validación de tokens OAuth2 en API Gateway
- TLS 1.2+ en todas las comunicaciones
- Firma de mensajes (opcional)
- Rate limiting
- Protección contra ataques (WAF)

6.3.7. Alta disponibilidad y resiliencia

Se considera las siguientes estrategias para lograr una alta disponibilidad y resiliencia:

CAPACIDAD	ESTRATEGIA
Disponibilidad	• API Gateway en alta disponibilidad • Balanceadores de carga • Reintentos automáticos • Circuit breakers
Resiliencia	• Persistencia de mensajes • Reintentos • Dead Letter Queues (DLQ)

6.3.8. Patrones de integración

PATRÓN	APLICACIÓN	JUSTIFICACIÓN
API Gateway	Actúa como punto único de entrada para todas las solicitudes provenientes de los canales frontend	• **Centralización de capacidades transversales:** Permite consolidar autenticación, autorización, rate limiting, logging y monitoreo en un único punto, reduciendo la complejidad en los microservicios. • **Desacoplamiento entre cliente y servicios internos:** El frontend no necesita conocer la estructura interna de los microservicios, lo que facilita la evolución de la arquitectura sin impactar a los consumidores. • **Mejora de la seguridad perimetral:** Permite implementar controles de seguridad avanzados (validación de tokens OAuth2, WAF, throttling) antes de que las solicitudes lleguen a los servicios internos.
Adapter	Encapsula la lógica de comunicación con sistemas externos como el Core bancario y sistemas complementarios	• **Aislamiento de sistemas legacy:** Permite desacoplar la lógica interna de la solución respecto a tecnologías, protocolos o cambios en sistemas externos. • **Facilidad de mantenimiento y evolución:** Si cambia el Core bancario o un proveedor externo, solo se modifica el adaptador sin afectar los microservicios. • **Normalización de interfaces:** Permite transformar diferentes formatos de entrada / salida en contratos homogéneos para el backend.
Evet – Driven	Notificaciones basado en la publicación y consumo de eventos mediante un bus de mensajería.	• **Desacoplamiento temporal y funcional:** Los servicios no dependen de respuestas inmediatas, lo que reduce la dependencia directa entre componentes. • **Alta escalabilidad:** Permite manejar grandes volúmenes de eventos (ej. notificaciones, auditoría) sin afectar el rendimiento de los servicios principales.

		• **Resiliencia ante fallos:** Los eventos pueden persistirse y reprocesarse, evitando pérdida de información en caso de fallos.
Circuit Breaker	Resiliencia, Evita llamadas repetitivas a servicios que están fallando, protegiendo la estabilidad del sistema	• **Prevención de fallos en cascada:** Si un sistema externo falla, el circuito se abre evitando afectar otros servicios. • **Mejora de la resiliencia del sistema:** Permite implementar respuestas controladas mientras el servicio se recupera. • **Protección de recursos:** Reduce el consumo innecesario de recursos ante fallos persistentes.
Retry Pattern	Recuperación, permite reintentar automáticamente operaciones fallidas bajo ciertas condiciones.	• **Recuperación ante fallos transitorios:** Muchas fallas en sistemas distribuidos son temporales (latencia, timeouts), por lo que un reintento puede resolverlas. • **Mejora de la confiabilidad del sistema:** Incrementa la probabilidad de éxito en operaciones críticas como transferencias. • **Optimización de la experiencia del usuario:** Reduce la necesidad de intervención manual ante errores momentáneos

6.3.9. Decisiones arquitectónicas

DECISIÓN	JUSTIFICACIÓN
Microservicios	Escalabilidad + desacoplamiento
Event – driven	Resiliencia + auditoría
CQRS	Optimización lectura / escritura
API Gateway	Seguridad centralizada
Cache (Redis)	Mejora de rendimiento

6.4. Datos

La capa de datos se diseña como un componente estratégico que garantiza la integridad, disponibilidad y trazabilidad de la información, soportando operaciones críticas del negocio bancario, optimizando el rendimiento mediante patrones avanzados y cumpliendo con los requerimientos regulatorios del sector financiero.

6.4.1. Descripción general

La capa de datos es responsable de la gestión, almacenamiento, acceso y procesamiento de la información utilizada por la solución de banca por internet de la entidad BP, se diseña bajo principios de: alta disponibilidad, consistencia y trazabilidad, escalabilidad, y seguridad de la información. Esta capa soporta:

- Persistencia de datos transaccionales
- Consulta de información de clientes y movimientos
- Registro de auditoría de todas las acciones
- Optimización de consultas para clientes frecuentes
- Integración con sistemas Core y complementarios

6.4.2. Tipo de datos gestionados

La solución maneja los siguientes tipos de datos:

- **Datos transaccionales**: transferencias, pagos
- **Datos de consulta**: movimientos históricos
- **Datos de cliente**: provenientes del core y sistema complementario
- **Datos de auditoría**: registro de acciones del usuario
- **Datos cacheados**: consultas frecuentes

6.4.3. Arquitectura de persistencia

MECANISMO	TIPO	USO
Base de datos transaccional	Relacional	• Transferencias • Pagos • Estados de transacción
Base de datos de auditoría	Append – only (inmutable)	• Registros de eventos del sistema • Trazabilidad completa
Base de datos de lectura	Optimizada para consultas	• Movimientos históricos • Consultas rápidas
Cache distribuido	Tecnología: Redis	• Clientes frecuentes • Consultas repetitivas

6.4.4. Patrones de datos

PATRÓN	APLICACIÓN	JUSTIFICACIÓN
CQRS (Command Query Responsibility Segregation)	Separación entre operaciones de escritura (commands) y lectura (queries): • **Escritura:** transferencias y pagos • **Lectura:** movimientos y consultas	• **Optimización del rendimiento:** Permite diseñar modelos de lectura altamente eficientes sin afectar la lógica transaccional. • **Escalabilidad independiente:** Las cargas de lectura y escritura pueden escalarse de forma separada. • **Mejor experiencia de usuario:** Consultas rápidas para información histórica.
Event Sourcing	Persistencia basada en eventos en lugar de estados finales • Auditoría • Registro de acciones del usuario	• **Trazabilidad completa:** Permite reconstruir todas las acciones realizadas por el cliente. • **Cumplimiento normativo:** Garantiza no repudio y auditoría detallada. • **Análisis posterior:** Permite uso en analítica o detección de fraude.
Cache Aside Pattern	El sistema consulta primero el cache antes de acceder a la base de datos • Clientes frecuentes • Consultas de movimientos	• **Reducción de latencia:** Mejora tiempos de respuesta en consultas repetitivas. • **Disminución de carga en sistemas Core:** Evita consultas innecesarias a sistemas legacy.

		• **Escalabilidad:** Permite manejar alto volumen de consultas concurrentes.
Database per Service	Cada microservicio gestiona su propia base de datos	• **Desacoplamiento total:** Evita dependencias entre servicios. • **Autonomía de equipos:** Permite evolución independiente. • **Escalabilidad:** Cada servicio escala según su carga.

6.4.5. Estrategia de auditoría

MODELO	COMPONENTES	FLUJO	JUSTIFICACIÓN
Event Sourcing + Event Store	• Event Bus • Servicio de Auditoría • Base de datos de auditoría	• Usuario ejecuta acción (ej. transferencia) • Servicio genera evento • Evento se publica en bus • Servicio de auditoría lo persiste	• Inmutabilidad • Trazabilidad completa • Capacidad de reprocesamiento

6.4.6. Persistencia para clientes frecuentes

SOLUCIÓN	COMPONENTES	JUSTIFICACIÓN
Cache + Read Model	• Redis • Base de datos de lectura	• **Optimización de consultas frecuentes:** Reduce tiempos de respuesta significativamente. • **Reducción de carga en sistemas externos:** Minimiza dependencias del Core bancario. • **Mejora de experiencia del usuario:** Respuestas rápidas en canales digitales.

6.4.7. Integración con sistemas core

ESTRATEGIA	JUSTIFICACIÓN
• No replicar completamente datos del Core • Uso de cache controlado • Uso de vistas consolidadas	• **Consistencia de datos:** El Core sigue siendo la fuente de verdad. • **Reducción de duplicidad:** Evita inconsistencias entre sistemas.

6.4.8. Seguridad de la información

Para garantizar la seguridad de la información, se implementa los siguientes controles:

- Encriptación en reposo (AES-256)
- Encriptación en tránsito (TLS 1.2+)

- Control de acceso a datos (RBAC)
- Tokenización de datos sensibles

6.4.9. Alta disponibilidad y recuperación

CARACTERÍSTICA	ESTRATEGIA
Disponibilidad	• Replicación de bases de datos • Multi-AZ • Backups automáticos • Failover automático
Recuperación ante desastres	• Replicación en otra región • Estrategia RPO/RTO definida

6.4.10. Decisiones arquitectónicas

DECISIÓN	JUSTIFICACIÓN
CQRS	Optimización lectura / escritura
Event Sourcing	Auditoría completa
Redis	Mejora de rendimiento
DB por servicio	Desacoplamiento
No replicar core	Consistencia

6.5. Seguridad transversal

La capa de seguridad se define como un componente transversal crítico que garantiza la protección integral de la solución bancaria, asegurando la confidencialidad, integridad y disponibilidad de la información, cumpliendo con los estándares regulatorios y mitigando riesgos asociados a amenazas internas y externas.

6.5.1. Descripción general

La capa de seguridad define el conjunto de controles, políticas y mecanismos que garantizan la protección integral de la solución de banca por internet de la entidad BP. Esta capa se implementa de forma **transversal**, abarcando todas las capas de la arquitectura (Frontend, Backend, Integración y Datos); cubriendo:

- Autenticación y autorización de usuarios
- Protección de datos sensibles
- Seguridad en comunicaciones
- Prevención de fraudes
- Cumplimiento normativo

6.5.2. Principios de seguridad

Se consideran los siguientes principios de seguridad:

- Seguridad por diseño
- Confianza cero
- Defensa en profundidad
- Menor privilegio
- Seguridad basada en identidad
- Trazabilidad y no repudio

6.5.3. Arquitectura de autenticación y autorización

Se utiliza la siguiente arquitectura de autenticación y autorización para las dos aplicaciones: SPA y aplicación móvil

ESTÁNDAR / PROCESO	FLUJO	MECANISMOS	JUSTIFICACIÓN
• OAuth 2.0 • OpenID Connect	Authorization Code + PKCE	• Usuario y contraseña • Autenticación multifactor (MFA) • Biometría (huella / reconocimiento facial)	• **Mayor seguridad en clientes públicos:** Evita exposición de secretos y protege contra interceptación de códigos. • **Cumplimiento de estándares modernos:** Recomendado para aplicaciones web y móviles.
• Onboarding seguro	• Validación de identidad mediante biometría facial • Integración con servicios especializados (AWS Rekognition o Azure Face API) • Creación de identidad digital segura	• Reconocimiento facial	

6.5.4. Gestión de identidad de acceso

COMPONENTES	JUSTIFICACIÓN
• Identity Provider (IdP) • Gestión de roles (RBAC) • Control de acceso basado en atributos	• **Control granular de accesos:** Permite definir permisos según tipo de usuario. • **Escalabilidad organizacional:** Facilita gestión de usuarios y roles en crecimiento.

6.5.5. Seguridad en la comunicación

Para garantizar la seguridad en la comunicación, se implementan los siguientes controles que proporcionan protección contra intercepciones de datos conforme establecen los estándares financieros:

- HTTPS obligatorio
- TLS 1.2 o superior
- Certificados digitales
- Comunicación segura entre microservicios

6.5.6. Seguridad en APIs

APLICACIÓN	CONTROLES	JUSTIFICACIÓN
En API Gateway	• Validación de tokens OAuth2 • Rate limiting • Throttling • Protección contra ataques (WAF)	• Prevención de ataques comunes (OWASP) • Protección de servicios críticos
Protección adicional	• Validación de inputs • Sanitización de datos • Manejo de errores controlado	

6.5.7. Seguridad de datos

APLICACIÓN	DESCRIPCIÓN	JUSTIFICACIÓN
Controles	• Encriptación en tránsito (TLS) • Encriptación en reposo (AES-256) • Tokenización de datos sensibles	• Protección de información sensible del cliente • Cumplimiento normativo (datos personales)
Gestión de claves	• Uso de servicios KMS (Key Management Service)	

6.5.8. Auditoría y no repudio

CONTROLES	JUSTIFICACIÓN
• Registro de eventos (Event Sourcing) • Logs firmados digitalmente • Trazabilidad completa	• Cumplimiento regulatorio • Capacidad de auditoría y trazabilidad

6.5.9. Seguridad en infraestructura

CONTROLES	JUSTIFICACIÓN
• WAF (Web Application Firewall) • Segmentación de red (VPC) • Subredes públicas y privadas • Security Groups / Firewalls • Bastion Host • Acceso restringido	• Protección perimetral • Aislamiento de componentes críticos

6.5.10. Monitoreo y detección de amenazas

COMPONENTES	CASOS DE USO	JUSTIFICACIÓN
• SIEM (Security Information and Event Management) • Monitoreo de logs • Alertas en tiempo real	• Detección de accesos sospechosos • Intentos fallidos de autenticación • Comportamientos anómalos	• Detección temprana de incidentes • Cumplimiento de estándares de seguridad

6.5.11. Prevención de fraude

ESTRATEGIA	JUSTIFICACIÓN
• Análisis de comportamiento • Validación de transacciones • Reglas antifraude	• Protección de transacciones financieras • Reducción de riesgos operativos

6.5.12. Alta disponibilidad y seguridad

ESTRATEGIA	JUSTIFICACIÓN
• Arquitectura Multi-AZ • Failover automático • Balanceadores de carga	• Continuidad del negocio • Disponibilidad de servicios críticos

6.5.13. Decisiones arquitectónicas

DECISIÓN	JUSTIFICACIÓN
OAuth2 + PKCE	Seguridad de clientes públicos
MFA	Reducción de riesgo de acceso
mTLS	Seguridad interna
KAMS	Gestión segura de claves
SIEM	Monitoreo centralizado

7. Arquitectura Cloud

Se propone el uso de una arquitectura basada en servicios cloud (AWS como referencia), esto permitirá una alta disponibilidad (Multi – AZ), escalabilidad automática y pago por consumo.

COMPONENTE	SERVICIO AWS
API Gateway	Amazon API Gateway
Microservicios	Amazon EKS / ECS
Autenticación	Amazon Cognito
Bases de datos	Amazon RDS / DynamoDB
Cache	Amazon ElastiCache (Redis)
Mensajería	Amazon SNS / SQS
Auditoría	CloudWatch + EventBridge
Seguridad	AWS WAF + Shield

8. Costos

Se establece estrategias para optimizar costos basadas en los siguientes principios: uso de servicios administrativos, escalabilidad automática; y, pago por consumo:

ESTRETEGIA	DESCRIPCIÓN / JUSTIFICACIÓN
Escalado automático	Ajuste dinámico de recursos según demanda
Uso de cache	Reduce consumo de APIs y bases de datos
Arquitectura Serverless (parcial)	Para eventos, auditoría y notificaciones
Uso de CDN	Reduce carga en backend

9. Modelo C4

9.1. Diagrama de contexto

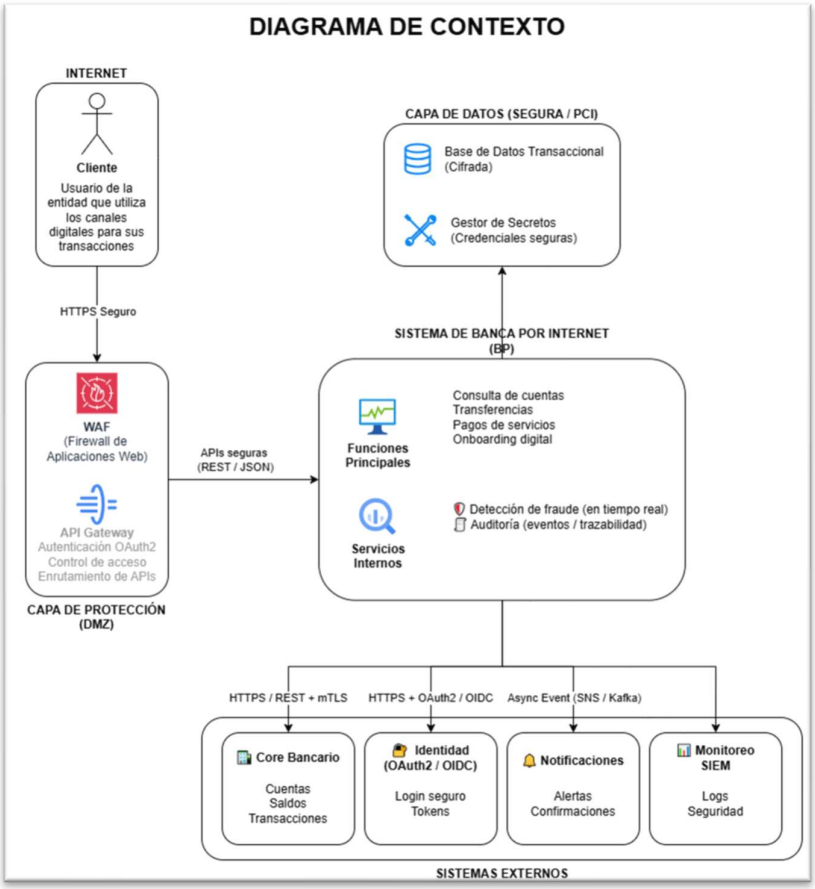

Figura 1. Diagrama de Contexto – Banca por Internet (BP)

El diagrama de contexto muestra la interacción del usuario con el sistema de banca por internet, así como la integración con sistemas externos críticos como el Core Bancario, el proveedor de autenticación y los servicios de notificación. Se evidencia que el sistema Banca por Internet (BP) actúa como intermediario central, gestionando las solicitudes del cliente y coordinando la comunicación con los sistemas internos y externos mediante interfaces seguras.

9.2. Diagrama de contenedores

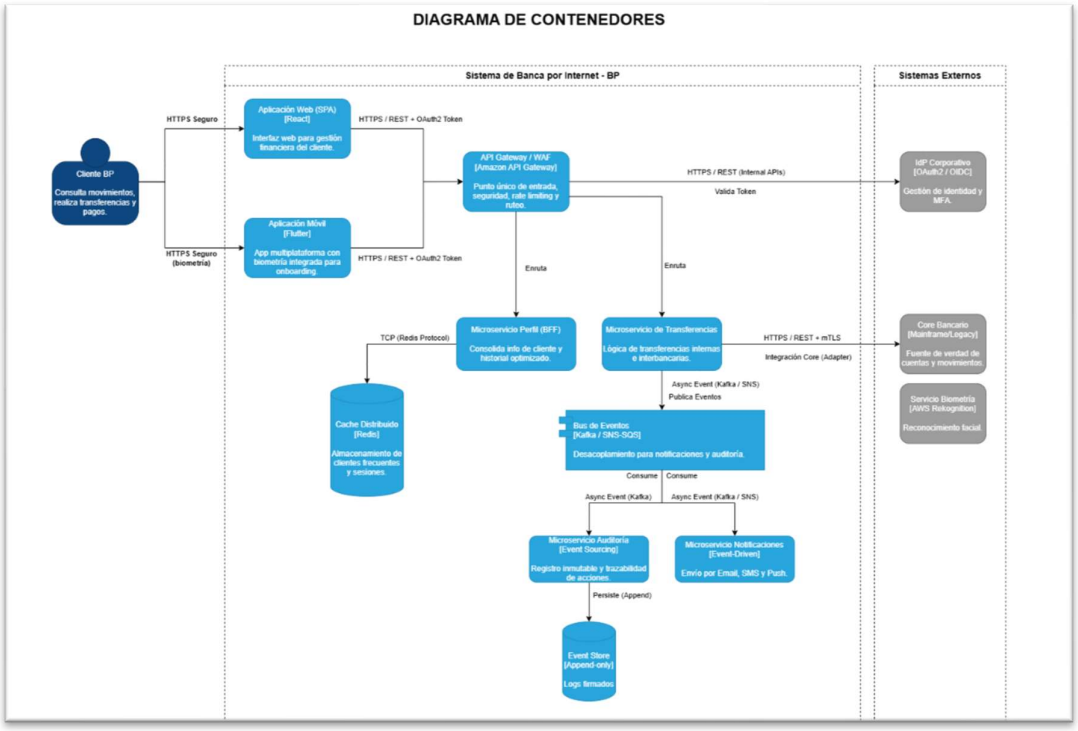

Figura 2. Diagrama de Contenedores – Banca por Internet (BP)

El diagrama de contenedores presenta la estructura de alto nivel de la solución, donde se identifican claramente los canales digitales (SPA y aplicación móvil), la capa de acceso mediante API Gateway, los microservicios backend y los mecanismos de integración. Se observa una arquitectura desacoplada, donde cada componente cumple una responsabilidad específica y se comunica mediante protocolos seguros (HTTPS) y eventos asíncronos.

9.3. Diagrama de componentes

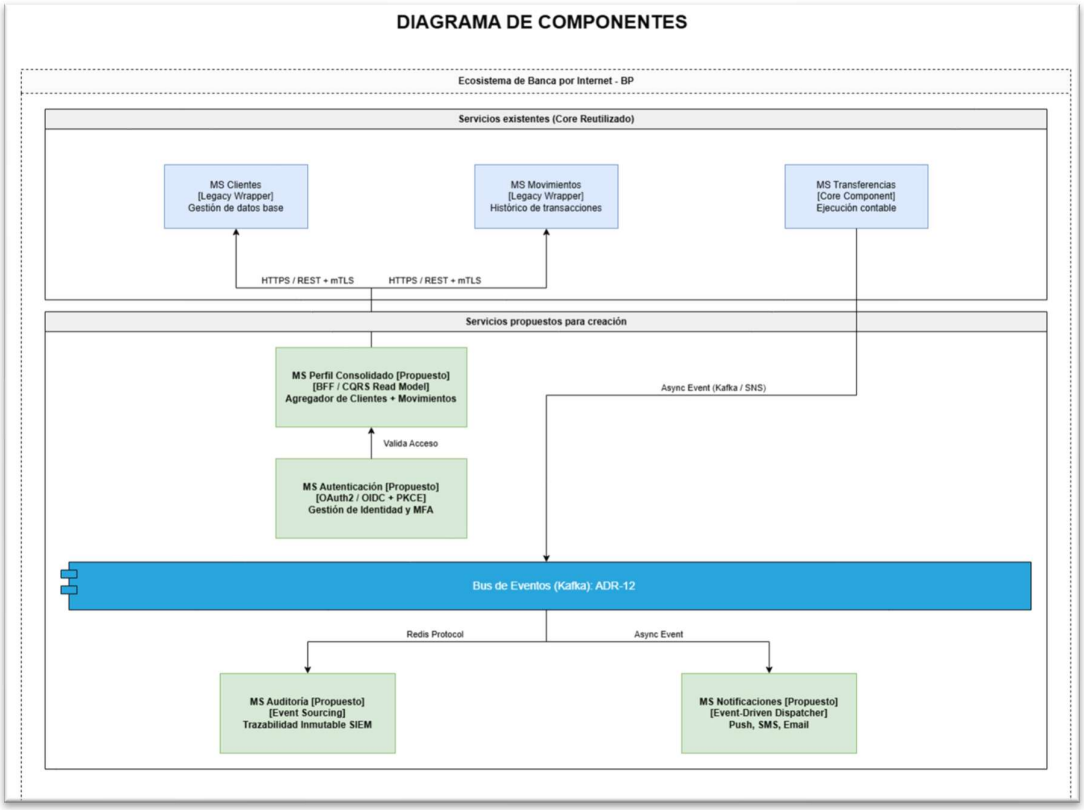

Figura 3. Diagrama de Componentes – Banca por Internet (BP)

El diagrama de componentes detalla la estructura interna de los microservicios, mostrando la separación entre lógica de negocio, adaptadores de integración y mecanismos de publicación de eventos. Se evidencia el uso de patrones como Adapter y Event-Driven, lo que permite desacoplar la lógica interna de las dependencias externas y mejorar la mantenibilidad del sistema.

10. Flujos End – to – End

10.1. Flujo de transferencia bancaria

El siguiente es el flujo de transferencias bancarias, se caracteriza por ser: seguro (OAuth2 + TLS); auditado (event sourcing); resiliente (event - driven); y, escalable (microservicios).

1. El usuario inicia una transferencia desde la aplicación (web o móvil).
2. El frontend envía la solicitud al API Gateway incluyendo el token de autenticación.
3. El API Gateway valida el token mediante el proveedor OAuth2.
4. La solicitud es redirigida al servicio de transferencias.
5. El servicio valida reglas de negocio (saldo, cuentas, límites).
6. Se invoca el Core bancario mediante un adaptador.

7. El Core procesa la transacción y devuelve el resultado.
8. El servicio de transferencias genera un evento de transacción.
9. El evento se publica en el bus de eventos.
10. El servicio de notificaciones envía alertas al cliente (email/SMS).
11. El servicio de auditoría registra la operación de forma inmutable.
12. El resultado final es retornado al cliente.

10.2. Flujo de consulta de movimientos

El siguiente es el flujo de consulta de movimientos, se caracteriza por ser: altamente eficiente (uso de cache); desacoplado (uso de adaptadores); auditado (registro de consulta); y, escalable (lectura optimizada con CQRS)

1. El usuario accede a la opción de consulta de movimientos desde la aplicación web o móvil.
2. El frontend envía la solicitud al API Gateway incluyendo el token de autenticación.
3. El API Gateway valida el token con el proveedor OAuth2.
4. La solicitud es redirigida al servicio de movimientos.
5. El servicio de movimientos consulta el cache (Redis) para verificar si existe información reciente.
6. Si la información está en cache, se retorna inmediatamente al frontend.
7. Si no está en cache, el servicio consulta el Core bancario mediante un adaptador.
8. El Core retorna los movimientos históricos.
9. El servicio transforma y normaliza los datos.
10. Los datos se almacenan temporalmente en cache para futuras consultas.
11. El servicio retorna la información al frontend.
12. Se genera un evento de consulta para fines de auditoría.
13. El servicio de auditoría registra la operación.

10.3. Flujo de onboarding digital

El siguiente es el flujo de onboarding digital con reconocimiento facial, se caracteriza por ser:

1. El usuario inicia el proceso de registro desde la aplicación móvil o web.
2. El frontend solicita permisos para captura de datos biométricos (rostro).
3. El usuario realiza la captura facial mediante la aplicación.
4. El frontend envía la información biométrica al servicio de onboarding a través del API Gateway.
5. El API Gateway valida la solicitud y el canal de comunicación (TLS).
6. El servicio de onboarding envía los datos a un proveedor externo de validación biométrica.
7. El proveedor valida la identidad comparando contra documento oficial o base de datos.
8. El proveedor retorna el resultado de la validación (exitosa o fallida).
9. Si la validación es exitosa, el sistema procede con la creación del perfil del cliente.
10. Se registra la identidad digital en el sistema de autenticación (IdP / OAuth2).
11. Se generan credenciales seguras para el usuario.
12. Se publica un evento de onboarding exitoso en el bus de eventos.
13. El servicio de notificaciones envía confirmación al usuario (email/SMS).

14. El servicio de auditoría registra todo el proceso de onboarding.
15. El sistema retorna confirmación al frontend.

11. Resumen de decisiones arquitectónicas (ADR Global)

11.1. Introducción

El presente apartado resume las principales decisiones arquitectónicas adoptadas en el diseño de la solución de banca por internet para la entidad BP. Cada decisión ha sido tomada considerando:

- Requerimientos funcionales del sistema
- Restricciones del ejercicio
- Buenas prácticas de arquitectura de software
- Cumplimiento normativo del sector financiero
- Necesidad de escalabilidad, resiliencia y seguridad

Las decisiones arquitectónicas adoptadas permiten construir una solución:

- Escalable
- Resiliente
- Segura
- Desacoplada
- Apegada a la normativa bancaria

11.2. Decisiones arquitectónicas claves

ADR	DECISIÓN	JUSTIFICACIÓN	ALTERNATIVAS EVALUADAS
ADR-01: Arquitectura basada en microservicios	Adoptar una arquitectura de microservicios para el backend	• **Escalabilidad independiente:** Permite escalar servicios críticos como transferencias sin afectar otros componentes. • **Desacoplamiento funcional:** Facilita la evolución independiente de cada dominio (clientes, movimientos, pagos)	**Monolito:** descartado por baja flexibilidad y escalabilidad limitada
ADR-02: Uso de API Gateway como punto único de entrada	Implementar un API Gateway para centralizar el acceso a los servicios backend	• **Centralización de seguridad:** Permite gestionar autenticación, autorización y rate limiting en un solo punto. • **Simplificación del cliente:** El frontend no necesita conocer la estructura interna de los servicios	**Acceso directo a microservicios:** descartado por mayor acoplamiento y riesgo de seguridad
ADR-03: Arquitectura basada en eventos (Event-Driven)	Adoptar un modelo de comunicación asíncrona basado en eventos	• **Desacoplamiento entre servicios:** Reduce dependencias directas entre componentes. • **Cumplimiento de requisitos de auditoría y notificación:** Permite procesar eventos de forma independiente (auditoría, notificaciones).	**Integración síncrona exclusiva:** descartada por baja resiliencia

ADR-04: Implementación de CQRS	Separar las operaciones de lectura y escritura	• **Optimización del rendimiento:** Permite diseñar modelos de lectura altamente eficientes. • **Escalabilidad independiente:** Las consultas y transacciones pueden escalarse por separado	**Modelo CRUD tradicional:** descartado por limitaciones de rendimiento
ADR-05: Uso de Event Sourcing para auditoría	Persistir eventos en lugar de estados para auditoría	• **Trazabilidad completa:** Permite reconstruir todas las acciones del usuario. • **Cumplimiento normativo:** Soporta requerimientos de auditoría y no repudio.	**Logs tradicionales:** descartados por menor capacidad de reconstrucción
ADR-06: Uso de Redis para cache (Cache Aside)	Implementar cache distribuido para clientes frecuentes	• **Reducción de latencia:** Mejora tiempos de respuesta en consultas repetitivas. • **Disminución de carga en sistemas Core:** Reduce dependencia de sistemas legacy.	**Sin cache:** descartado por impacto en rendimiento
ADR-07: Uso de OAuth 2.0 + OpenID Connect	Delegar autenticación en un proveedor basado en OAuth2	• **Cumplimiento de estándares de seguridad:** Protocolo ampliamente adoptado en banca. • **Separación de responsabilidades:** Evita implementar lógica de autenticación en el backend.	**Autenticación propia:** descartada por riesgos de seguridad
ADR-08: Uso de Authorization Code + PKCE	Adoptar este flujo para SPA y mobile	• Protección contra interceptación de tokens • Recomendado para clientes públicos	
ADR-09: Aplicación móvil en framework multiplataforma (Flutter)	Desarrollar la app móvil en Flutter	• Reducción de costos de desarrollo • Soporte robusto para biometría	**Desarrollo nativo:** descartada por mayor costo y tiempo
ADR-10: Uso de adaptadores para integración con sistemas externos	Implementar adaptadores para Core y sistemas externos	• Aislamiento de sistemas legacy • Facilidad de mantenimiento	
ADR-11: Base de datos por microservicio	Cada servicio gestiona su propia base de datos	• Desacoplamiento total • Escalabilidad independiente	
ADR-12: Uso de mensajería (Kafka / SNS-SQS)	Implementar un bus de eventos para comunicación asíncrona	• Alta resiliencia • Procesamiento desacoplado	
ADR-13: Arquitectura desplegada en nube (AWS/Azure)	Utilizar infraestructura cloud	• Alta disponibilidad (Multi-AZ) • Escalabilidad automática	
ADR-14: Implementación de WAF y seguridad perimetral	Proteger la solución mediante WAF y controles de red	• Protección contra ataques externos • Cumplimiento de estándares de seguridad	
ADR-15: Implementación de SIEM para monitoreo	Centralizar eventos de seguridad	• Detección temprana de amenazas • Cumplimiento normativo	

12. Trade – offs Aquitectónico

La arquitectura propuesta incorpora diversas decisiones que implican compromisos (trade-offs) entre diferentes atributos de calidad como rendimiento, escalabilidad, complejidad y mantenibilidad.

A continuación, se detallan los principales trade-offs considerados:

TRADE - OFF	DECISIÓN	VENTAJAS	DESVENTAJAS	CONCLUSIÓN
Microservicios vs Monolito	Microservicios	• Escalabilidad independiente • Desacoplamiento funcional	• Mayor complejidad operativa • Necesidad de orquestación y monitoreo distribuido	Se prioriza escalabilidad y flexibilidad sobre simplicidad, alineado a un entorno bancario de alta demanda.
Comunicación Síncrona vs Asíncrona	Arquitectura híbrida (REST + eventos)	• Respuesta inmediata en operaciones críticas • Desacoplamiento en procesos secundarios	• Mayor complejidad de implementación • Necesidad de manejo de consistencia eventual	Se equilibra experiencia del usuario y resiliencia del sistema.
CQRS vs Modelo CRUD	CQRS	• Optimización de consultas • Escalabilidad independiente	• Complejidad en sincronización de datos • Mayor esfuerzo de implementación	Se prioriza rendimiento y experiencia de usuario.
Event Sourcing vs Logging Tradicional	Event Sourcing	• Trazabilidad completa • Soporte para auditoría y no repudio	• Trazabilidad completa • Soporte para auditoría y no repudio	Se prioriza cumplimiento normativo y auditoría.
Cache vs Consistencia Fuerte	Uso de cache (Redis)	• Reducción de latencia • Mejor experiencia de usuario	• Riesgo de datos desactualizados	Se acepta consistencia eventual en favor de rendimiento.